Day 141 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 2 reference

Watch: Denoising Diffusion Probabilistic Models from Scratch using PyTorch! (Papers in 100 Lines of Code) | Generative Image Diffusion from Scratch! PyTorch Deep Learning Tutorial (Luke Ditria)

`day141_tiny_diffusion.py`, segment by segment, in the order `main()` uses things. The straight-line guess, the three readings of the network's output, the image training loop, the evaluation, the nearest-image search and the sample scorer are new; the schedule, time embedding and reverse_sample are Day 140's parts, the scorer is Day 138's, and the network code and Adam are Day 126's, all imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: t = step (1–1,000) · A_t = ABAR[t] · ε = noise · x_0 = clean digit, pixels in [–1, 1] · C = c(t) for every t

Segment 0 — Setup

Imports: this week's folder and four earlier ones

CODE

```
sys.path.insert(0, HERE)
for w in ("week-20-generative-models-gans", "week-19-unsupervised-learning-ii",
    "week-18-unsupervised-learning-i", "week-17-classical-ml-kernels-ensembles"):
    sys.path.insert(0, os.path.join(HERE, "..", w))
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day126_autoencoders as d126m
    import day139_gan_build_and_forward_diffusion as d139
    import day140_exercise as d140
```

- `sys.path.insert(0, HERE)` — the script's own folder goes first on the search path, so Day 140's part in the same week is found.
- `with contextlib.redirect_stdout(io.StringIO()):` — everything printed inside the block goes into a string that is thrown away: the imported parts run their own checks quietly.

Constants

CODE

```
BETAS, ABAR = d140.BETAS, d140.ABAR
SIZES = [816, 512, 512, 784]
ACTS = ["relu", "relu", "linear"]
STEPS = 30000
CHECKS = (2500, 5000, 10000, 20000, 30000)
```

- `BETAS, ABAR = d140.BETAS, d140.ABAR` — two names bound at once to Day 140's arrays; nothing is copied.
- `["relu", "relu", "linear"]` — one activation name per layer, the form Day 126's forward expects.
- `CHECKS = (...)` — a tuple: `s in CHECKS` asks whether the current step is one of them.

Segment 1 — The model

skip_coef: c(t) for all 1,001 steps at once

MATH

$$c(t) = \sqrt{(1 - A_t) / (A_t \, E[x_0^2] + 1 - A_t)}$$

CODE

```
def skip_coef(X2):
    m2 = float((X2.astype(np.float64) ** 2).mean())
    return np.sqrt(1 - ABAR) / (ABAR * m2 + 1 - ABAR)
```

- `float((X2.astype(np.float64) ** 2).mean())` — the average squared pixel, in double precision, as a plain Python number.
- `np.sqrt(1 - ABAR) / (ABAR * m2 + 1 - ABAR)` — ABAR is an array, so this is computed for every t in one line: C[t] then looks one up.

guess: one network, three readings of its output

MATH

$$x'_0 = f(x_t, t) \Rightarrow \varepsilon' = (x_t - \sqrt{A_t}\, x'_0) / \sqrt{(1 - A_t)}; \quad \varepsilon' = f(x_t, t) \Rightarrow x'_0 = (x_t - \sqrt{(1 - A_t)}\, \varepsilon') / \sqrt{A_t}$$

CODE

```
def guess(net, xt, t, kind="x0", C=None):
    A = d126.forward(net, np.concatenate([xt, d140.time_embed(t)], 1).astype(np.float32), ACTS)
    o = A[-1].astype(np.float64); sa, sb = np.sqrt(ABAR[t])[:, None], np.sqrt(1 - ABAR[t])[:,
        None]
    if kind == "x0":
        x0 = o; e = (xt - sa * x0) / sb
    else:
        e = o + (C[t][:, None] * xt if kind == "eps+line" else 0.0); x0 = (xt - sb * e) / sa
    return A, e, x0
```

- `np.concatenate([xt, d140.time_embed(t)], 1)` — 784 pixels and 32 time numbers side by side: 816 inputs per row.
- `np.sqrt(ABAR[t])[:, None]` — one value per row (t is an array), turned into a column so it multiplies every pixel of that row.
- `if kind == "x0":` — the output is the clean digit; the noise is then computed from it.
- `(C[t][:, None] * xt if kind == "eps+line" else 0.0)` — a conditional expression: add c(t) x_t only for that reading; adding 0.0 changes nothing.
- `return A, e, x0` — three results: every layer's output (needed for backward) and both guesses.

sampler_fn: a function that builds a function

CODE

```
def sampler_fn(net, kind="x0", C=None, clip=False):
    def eps_fn(x, t):
        _, e, x0 = guess(net, x, np.full(len(x), t), kind, C)
        if clip:
            x0 = np.clip(x0, -1, 1); e = (x - np.sqrt(ABAR[t]) * x0) / np.sqrt(1 - ABAR[t])
        return e
    return eps_fn
```

- `def eps_fn(x, t):` — a function defined inside another one; it remembers net, kind, C and clip from the call that made it (a closure).

- `_, e, x0 = guess(...)` — the underscore receives the layer outputs, which sampling does not need.
- `np.clip(x0, -1, 1)` — values below –1 become –1 and above 1 become 1.
- `return eps_fn` — the inner function itself is returned, not called: Day 140's reverse_sample calls it 1,000 times.

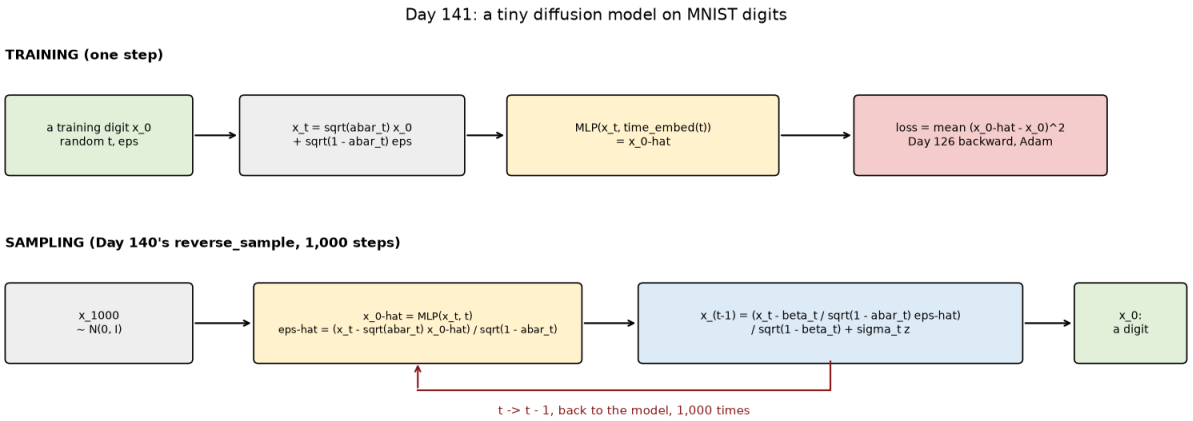
to_pixels

CODE

```
def to_pixels(S):
    return np.clip((S + 1) / 2, 0, 1).astype(np.float32)
```

- `(S + 1) / 2` — maps [–1, 1] back to [0, 1], the scale the scorer and the pictures use.

SYSTEM DIAGRAM



Training and sampling.

Segment 2 — Training and measuring

train_image_denoiser: Ho et al.'s loop on images

MATH

$$loss = mean\ |x'_0 - x_0|^2\ (or\ mean\ |\varepsilon' - \varepsilon|^2), \quad d\ loss\ /\ d\ output = 2\ (miss)\ /\ size$$

CODE

```
def train_image_denoiser(X2, steps, kind="x0", C=None, sizes=SIZES, zero_out=True, seed=0,
    lr=1e-3, batch=128,
                         every=1000, Xev=None, hook=None):
    net = d126.init_net(sizes, seed); opt = d126.Adam(net, lr)
    if zero_out:
        net[-1] = (net[-1][0] * 0, net[-1][1])
    rng = np.random.RandomState(seed); log = []; run = np.zeros(3)
    for s in range(1, steps + 1):
        x0 = X2[rng.randint(0, len(X2), batch)].astype(np.float64); t = rng.randint(1, len(BETAS)
            + 1, batch)
        e = rng.standard_normal(x0.shape); xt = np.sqrt(ABAR[t])[:, None] * x0 + np.sqrt(1 -
            ABAR[t])[:, None] * e
        A, eh, x0h = guess(net, xt, t, kind, C)
        miss = x0h - x0 if kind == "x0" else eh - e
        grads, _ = d126.backward(net, A, ACTS, (miss * 2 / miss.size).astype(np.float32))
        opt.step(net, grads)
        run += ((miss ** 2).mean(), (A[1] > 0).mean(), (A[2] > 0).mean())
        if s % every == 0:
            ev = eval_net(net, *Xev, kind=kind, C=C)[0 if kind == "x0" else 1] if Xev is not None
                else np.nan
            log.append((s, *(run / every), ev)); run[:] = 0
        if hook is not None:
            hook(s, net)
    return net, log
```

- `net[-1] = (net[-1][0] * 0, net[-1][1])` — the last layer's (weights, bias) pair is replaced by (zeros, bias): every first guess is 0.
- `run = np.zeros(3)` — three running sums in one array, added to together and reset together with run[:] = 0.
- `X2[rng.randint(0, len(X2), batch)]` — 128 random row numbers, with repeats allowed, pick a batch.
- `miss = x0h - x0 if kind == "x0" else eh - e` — the error in whatever the network was asked for.
- `(miss * 2 / miss.size).astype(np.float32)` — the gradient of the mean squared error, in the network's own precision.
- `*(run / every)` — the star spreads the three averages into the tuple being built.
- `eval_net(...)[0 if kind == "x0" else 1]` — eval_net returns (x_0 error, ε error); the index picks the matching one.
- `hook(s, net)` — a function passed in by the caller and called every step; section 3's hook scores samples at the CHECKS steps.

eval_net: both errors on fixed triples

CODE

```
def eval_net(net, x0, t, e, kind="x0", C=None):
    xt = np.sqrt(ABAR[t])[:, None] * x0 + np.sqrt(1 - ABAR[t])[:, None] * e; ex = ee = 0.0
    for i in range(0, len(t), 500):
        _, eh, x0h = guess(net, xt[i:i + 500], t[i:i + 500], kind, C)
        ex += ((x0h - x0[i:i + 500]) ** 2).sum(); ee += ((eh - e[i:i + 500]) ** 2).sum()
    return float(ex / x0.size), float(ee / e.size)
```

- `ex = ee = 0.0` — a chained assignment: both names start at 0.0.
- `for i in range(0, len(t), 500):` — batches of 500 keep the memory small; each batch may hold many different t.
- `x0[i:i + 500]` — slicing past the end is safe: the last slice is just shorter.

nearest_train: distances without a 2,000 x 60,000 loop

MATH

$$|a - b|^2 = |a|^2 + |b|^2 - 2\, a{\cdot}b$$

CODE

```
def nearest_train(F, Xtr, chunk=250):
    n2 = (Xtr.astype(np.float32) ** 2).sum(1); dist, idx = [], []
    for i in range(0, len(F), chunk):
        D = (F[i:i + chunk] ** 2).sum(1)[:, None] + n2[None] - 2 * F[i:i + chunk] @ Xtr.T
        j = D.argmin(1); idx.append(j); dist.append(np.sqrt(np.maximum(D[np.arange(len(j)), j],
            0)))
    return np.concatenate(dist), np.concatenate(idx)
```

- `(F[i:i + chunk] ** 2).sum(1)[:, None] + n2[None] - 2 * F[i:i + chunk] @ Xtr.T` — $|a - b|^2 = |a|^2 + |b|^2 - 2\, a{\cdot}b$ for every pair at once: one matrix product of 250 × 60,000.
- `D[np.arange(len(j)), j]` — fancy indexing with two arrays: row k, column j[k], the smallest distance in each row.
- `np.maximum(..., 0)` — rounding can make a tiny squared distance negative; clip it before the square root.

sample_scores

CODE

```
def sample_scores(net, sc, kind="x0", C=None, var="posterior", clip=False, n=2000):
    t0 = time.perf_counter(); F = to_pixels(d140.reverse_sample(sampler_fn(net, kind, C, clip), n,
        np.random.RandomState(1), var, d=784))
    return F, time.perf_counter() - t0, sc(F)
```

- `np.random.RandomState(1)` — the same starting noise every time, so different models and settings are compared on equal terms.
- `d=784` — Day 140's reverse_sample was written for 2-D points; d sets the size of each sample.
- `return F, time.perf_counter() - t0, sc(F)` — the timing is taken before scoring starts, so it covers sampling only.

Segment 3 — The experiments

demo_first_drafts: four readings in one loop

CODE

```
for name, kind, sizes in (("eps, 512 wide", "eps", SIZES), ("eps, 1,024 wide", "eps", [816, 1024,
    1024, 784]),
                          ("eps on top of c(t) x_t, 512 wide", "eps+line", SIZES), ("x_0 (the
                              clean digit), 512 wide", "x0", SIZES)):
    t0 = time.perf_counter(); net, _ = train_image_denoiser(X2[:60000], 3000, kind, C,
        sizes=sizes, every=3000); took = time.perf_counter() - t0
    F, _, (i, f, p, r, n) = sample_scores(net, sc, kind, C); out[name] = F[:8]
    print(f"{name:>34} {i:6.3f} {f:8.2f} {p:10.3f} {r:7.3f} {F.mean():6.3f} {took:5.0f}s")
```

- `for name, kind, sizes in ((...), (...), ...):` — a tuple of three-item tuples, unpacked into three names each time round.
- `F, _, (i, f, p, r, n) = sample_scores(...)` — nested unpacking: the third result is itself a 5-tuple of scores.
- `out[name] = F[:8]` — the first 8 samples of each draft, kept for the figure.

demo_train: a hook that scores samples during training

CODE

```
scores = []; t_sample = [0.0]
def hook(s, net):
    if s in CHECKS:
        _, took, r = sample_scores(net, sc); t_sample[0] += took; scores.append((s, *r))
t0 = time.perf_counter(); net, log = train_image_denoiser(X2[:60000], STEPS, Xev=Xev, hook=hook);
    total = time.perf_counter() - t0
```

- `t_sample = [0.0]` — a one-item list, so the inner function can add to it (t_sample[0] += ...) without a nonlocal statement.
- `scores.append((s, *r))` — the step number followed by the five scores, as one tuple.
- `hook=hook` — the inner function is passed to the training loop like any other value.

demo_sampling: two choices, six runs

CODE

```
for clip in (False, True):
    for var in ("posterior", "beta", "none"):
        F, took, (i, f, p, r, n) = sample_scores(net, sc, var=var, clip=clip); out[(var, clip)] =
            F
        said = np.concatenate([sc.head.forward(sc.feats(F[k:k + 1000])).argmax(1) for k in
            range(0, 2000, 1000)])
        print(f"{var:>10} {'yes' if clip else 'no':>5} {i:7.3f} {f:8.3f} {p:10.3f} {r:7.3f}
            {F.mean():6.3f} {took:5.0f}s   "
              f"{' '.join(str(c) for c in np.bincount(said, minlength=10))}")
```

- `out[(var, clip)] = F` — a tuple key: one entry per combination of settings.
- `.argmax(1)` — for each sample, the digit whose score is largest.
- `np.bincount(said, minlength=10)` — how many times each digit 0–9 appears, with 10 entries even if some digit never does.

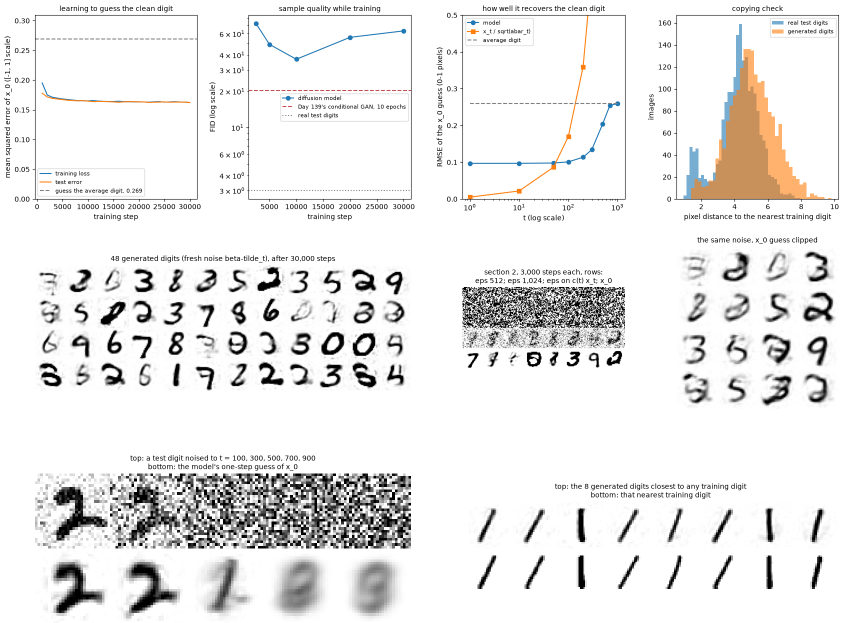
demo_copying

CODE

```
Xtr = X[:60000]; T = X[60000 + np.random.RandomState(0).permutation(10000)[:2000]]
ds, js = nearest_train(F, Xtr); dt, _ = nearest_train(T, Xtr)
print("pixel distance (0-1 pixels, 784 of them) from each image to its nearest of the 60,000
    training digits")
print(f"{'images':>22} {'min':>6} {'1%':>6} {'10%':>6} {'median':>7} {'mean':>6}")
for name, d in (("2,000 generated", ds), ("2,000 real test digits", dt)):
    print(f"{name:>22} {d.min():6.2f} {np.percentile(d, 1):6.2f} {np.percentile(d, 10):6.2f}
        {np.median(d):7.2f} {d.mean():6.2f}")
print(f"generated digits closer to a training digit than the closest real test digit is
    ({dt.min():.2f}): {int((ds < dt.min()).sum())}")
print(f"distinct nearest training digits among the 2,000 samples: {len(np.unique(js))}")
```

- `60000 + ...permutation(10000)[:2000]` — 2,000 test digits in random order: MNIST's stored test order is not random (Day 138).
- `np.percentile(d, 1)` — the value below which 1% of the distances fall.
- `int((ds < dt.min()).sum())` — a boolean array summed counts its True values.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `def f(): def g(): ... return g` — a closure: an inner function that remembers the outer call's values.
- `a = b = 0.0` — chained assignment.
- `x, _, (i, f) = ...` — nested unpacking; _ for an unused value.
- `box = [0.0]; box[0] += x` — a mutable box an inner function can update.
- `np.bincount(a, minlength=k)` — counts of 0, 1, ..., k – 1 in an integer array.
- `np.percentile(d, q)` — the q-th percentile of an array.

End of Day 141 syntax reference. For the experiments and what they showed, see the Day 141 complete guide.